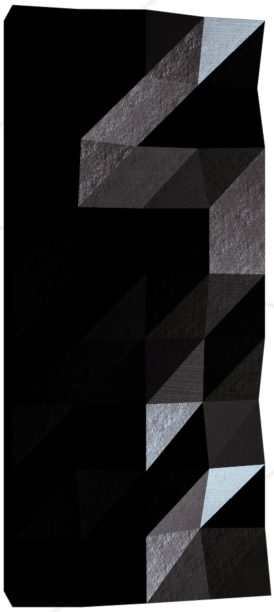

Swifty Protein

3D, fingerprints and other things

Summary: This project aims to introduce you to advanced mobile frameworks and application development.

Version: 6.0

Contents

I Forewords **2**

II Introduction **3**

III AI Instructions **4**

IV Goals **6**

V General instructions **8**

VI Mandatory part **10**
- VI.1 Application Icon and Launch Screen 10
- VI.2 Login View 10
- VI.3 Protein List View 11
- VI.4 Protein View 12

VII Bonus **14**
- VII.1 Multiple Visualization Models 14
- VII.2 Advanced User Interface 14
- VII.3 Enhanced Molecular Interactions 15
- VII.4 Performance and Caching 15
- VII.5 Extended Sharing and Export 16

VIII Turn-in and peer-evaluation **17**
- VIII.1 Evaluation Guidelines 17
- VIII.2 Common Pitfalls to Avoid 17

Chapter I

Forewords

Here's what wikipedia has to say about Pikachu :

Pikachu are a species of Pokemon, fictional creatures that appear in an assortment of comic books, animated movies and television shows, video games, and trading card games licensed by The Pokemon Company, a Japanese corporation. The Pikachu design was conceived by Ken Sugimori. Pikachu first appeared in Pokemon Red and Green in Japan, and later in the first internationally-released Pokemon video games Pokemon Red and Blue for the original Game Boy. Pikachu is considered to be the most emblematic pokemon from the starter pack.

Like other species of Pokemon, Pikachu are often captured and groomed by humans to fight other Pokemon for sport. Pikachu are one of the most well-known varieties of Pokemon, largely because a Pikachu is a central character in the Pokemon anime series. Pikachu is regarded as a major character of the Pokemon franchise as well as its mascot, and has become an icon of Japanese pop culture in recent years.

Chapter II

Introduction

The research lab `Noachlly Global Pharmacetics Drugs & Medicine Inc.` needs a protein visualizer to help the world understand its research. `Noachlly Global Pharmacetics Drugs & Medicine Inc.` works with the world-renowned Protein Data Bank (PDB), the global protein database.

Understanding the three-dimensional structure of proteins is crucial in modern pharmaceutical research and drug development. Proteins are the workhorses of biological systems, and their shape determines their function. By visualizing these molecular structures, researchers can better understand how drugs interact with their targets, leading to more effective treatments for diseases.

The RCSB Protein Data Bank is the world's leading repository of 3D structural data of large biological molecules, including proteins and nucleic acids. It contains over 200,000 structures determined by X-ray crystallography, NMR spectroscopy, and cryo-electron microscopy. For this project, you'll work specifically with ligands - small molecules that bind to proteins and are often the basis for drug design.

You have access to this database to build an app that allows you to visualize protein ligands using standardized molecular representations. This requires you to parse Chemical Information Files (.cif format), the modern standard that replaced the legacy .pdb format in 2014.

To complete this project, you need to use a framework appropriate for your platform (e.g. SceneKit for iOS, ViroCore or filament for Android, or other modern 3D rendering frameworks). These high-level 3D graphics frameworks will help you create 3D animated scenes and effects in your applications, incorporating physics engines, particle generators, and easy ways to script the actions of 3D objects.

`Noachlly Global Pharmacetics Drugs & Medicine Inc.` recruited you for your mobile development expertise, and now they're challenging you to go further by combining bioinformatics, 3D graphics, and modern mobile security features in this application.

Heal the world, make it a better place. For you. For me. For the entire universe.
Michel Jacques, CEO, Noachlly Global Pharmacetics Drugs & Medicine Inc.

Chapter III

AI Instructions

Context

AI is now a powerful coding partner — alongside your peers — for tackling large and demanding projects. You will guide it through both technical and non-technical aspects of your work.

AI tools can boost your efficiency and improve the quality of your output, but you should be able to dive deep into any part of the project without relying on them.

Your AI partner supports you, but you remain fully responsible for making informed technical decisions and to clearly explain and defend them.

Main message

- ☛ Strive for a mature and responsible use of AI.
- ☛ Never let AI take responsibility for decisions — especially when it lacks awareness of your goals, constraints, or team dynamics.
- ☛ Maintain creativity, innovation, and human oversight through active collaboration with your peers. AI is trained on existing data and rarely generates truly new ideas.
- ☛ Stay informed about emerging trends and be ready to adapt to new concepts and technologies.

Learner rules:

- Maintain intellectual leadership over your projects and make your own informed decisions.
- Prioritise the collective intelligence of your team and peers.
- Actively stay informed about the ongoing evolution of AI technologies.

Phase outcomes:

- AI engineering skills.
- Increased efficiency.
- Greater reliability and quality.
- A pioneering mindset.

Comments and examples:

- Your peers can identify trade-offs, question assumptions, and help you improve. The first answer from an AI might not be the best — it may lack efficiency, security, or real added value. Now more than ever, you should rely on your peers.
- AI can make you faster, but your peers make you better. Collaboration, discussion, and mutual challenge are key to success.
- Be transparent about how AI was used in your projects, and clearly identify what was generated by AI tools.

✓ Good practice:

> I asked AI to help generate unit tests for my API. I reviewed them with my teammate, and we adjusted them for edge cases. It saved time, and we both learned something new.

✗ Bad practice:

> I had AI generate the entire architecture of my project. It "works," but when I'm asked to explain design decisions during the peer review or in front from of a customer, I cannot. I lose credibility and I fail.

Chapter IV

Goals

This project aims to help you become familiar with several advanced mobile development concepts and scientific computing principles:

- **3D Graphics Rendering**: Learn to use high-level 3D frameworks (SceneKit on iOS, ViroCore/filament on Android) to render complex molecular structures. Understand scene graphs, cameras, lighting, and materials.
- **Biometric Authentication**: Implement secure authentication using modern biometric APIs (Touch ID/Face ID on iOS, BiometricPrompt on Android). Understand the security implications and fallback mechanisms.
- **Network Programming**: Fetch and parse data from remote servers (RCSB database). Handle network errors gracefully, implement proper loading states, and manage asynchronous operations.
- **File Parsing**: Parse the Chemical Information File (.cif) format, a structured text format containing atomic coordinates and molecular data. Extract relevant information for 3D visualization.
- **Data Structures**: Represent complex molecular structures in code, managing atoms, bonds, and their spatial relationships efficiently.
- **User Interface Design**: Create intuitive mobile interfaces with search functionality, list views, and responsive layouts that adapt to different screen sizes and orientations.
- **Social Sharing**: Integrate platform-specific sharing APIs to allow users to export and share molecular visualizations.
- **Biochemistry Fundamentals**: Understand basic molecular biology concepts including:
  - CPK (Corey-Pauling-Koltun) coloring scheme for atoms
  - Ball-and-stick molecular models
  - Atomic elements and their properties
  - Ligand structures and their role in drug design

- **Performance Optimization**: Render potentially complex 3D scenes smoothly on mobile devices, managing memory and computational resources effectively.

Chapter V

General instructions

- You must choose between iOS, Android, or a multiplatform solution.
- This project will be evaluated by human reviewers only.
- This project must be written using the latest SDK/IDE/language versions available for your chosen platform at the time of evaluation:
  - iOS: Latest Swift and Xcode versions
  - Android: Latest Kotlin/Java and Android Studio versions
  - Multiplatform: Latest Flutter, React Native, or similar framework
- This project must use the RCSB website to retrieve .cif files (Chemical Information File format). The .cif format is the modern RCSB standard since 2014, replacing the deprecated .pdb format.
- Ligands can be downloaded from: `https://files.rcsb.org/ligands/view/{ligand}.cif`
- The .cif format is text-based and easier to parse than .pdb for ligand structures. You must implement your own parser or use an appropriate library.

> The subject refers to the use of .cif files to represent protein structures. However, other open and publicly available file formats commonly used to represent proteins (such as .sdf) also exist and are widely used in practice. These alternative open formats are authorized, as long as they correctly represent protein structures and are compatible with the requirements of the project.

- The project must use modern layout techniques (Auto Layout/constraints on iOS, ConstraintLayout on Android) to ensure a responsive user interface that adapts to:
  - Different screen sizes (phones, tablets)
  - Different orientations (portrait, landscape)
  - Different screen densities
- Your application must handle all errors gracefully. Crashes, freezes, or unexpected behavior are unacceptable. This includes:

  - Network errors (no connection, timeouts, 404 errors)
  - Parsing errors (malformed .cif files)
  - Memory constraints (large molecules)
  - Invalid user input

- All network operations must be performed asynchronously on background threads. The UI must remain responsive at all times.
- You must provide clear user feedback for all operations:
  - Loading indicators during network requests
  - Error messages that are user-friendly and actionable
  - Success confirmations where appropriate
- Your application must not leak memory. Use appropriate memory management techniques for your platform (ARC on iOS, proper lifecycle management on Android).
- The application must be tested on real devices, not just simulators/emulators. Performance and behavior can differ significantly.
- Your code must be well-organized and follow platform-specific best practices:
  - iOS: Follow Apple's Human Interface Guidelines and Swift API Design Guidelines
  - Android: Follow Material Design Guidelines and Kotlin/Java coding conventions
- Security is paramount. You must:
  - Never store sensitive data in plain text
  - Use platform-specific secure storage (Keychain on iOS, KeyStore on Android)
  - Validate all data received from the network
  - Handle biometric authentication failures securely
- The application must work offline for previously loaded ligands (optional caching).
- Your application must be accessible and follow accessibility guidelines where possible (VoiceOver support, sufficient contrast ratios, etc.).
- `Any application that crashes, freezes, or displays errors during evaluation will receive a failing grade, regardless of implemented features.`

Chapter VI

Mandatory part

To validate the mandatory part, you must complete all of the following criteria. Each feature must work flawlessly, and the application must handle all edge cases gracefully.

VI.1 Application Icon and Launch Screen

- Choose an icon for your application that is in accordance with the theme (molecular/scientific/medical). The icon must be properly sized for all required resolutions on your platform.
- When the application is launched, it must display a Launch Screen that remains visible for at least 1-2 seconds. The launch screen should be professional and thematic.
- The launch screen must not be a static image that looks like the app is loading forever. It should be a clean, branded splash screen.

VI.2 Login View

- Choose an authentication system that allows you to store and manage user accounts. Options include:
  - Firebase Authentication
  - Platform-specific secure storage (Keychain on iOS, KeyStore on Android)
  - Custom backend with secure password hashing
- A user must be able to create an account with at least:
  - Username (unique identifier)
  - Password (must meet minimum security requirements)
- A user must be able to log in with biometric authentication:
  - iOS: Face ID or Touch ID
  - Android: BiometricPrompt API (fingerprint, face, iris)

- If biometric authentication fails, you must display a clear error message in a popup or alert dialog explaining why authentication failed (e.g., "Authentication failed. Please try again." or "Too many failed attempts.").
- If the device does not support biometric authentication, the user must be able to log in with their username and password. The biometric option should be hidden or disabled in this case.
- **Security Requirement**: The Login View must ALWAYS be displayed when launching the app. This means:
  - When the app is first launched
  - When the app is brought back from the background
  - When the app is reopened after pressing the Home button
  - Even if the user was previously authenticated

  This ensures that sensitive molecular data is protected.
- Passwords must never be stored in plain text. Use appropriate hashing algorithms (bcrypt, Argon2, or platform-specific secure storage).
- The login interface should be user-friendly with clear labels, appropriate keyboard types, and helpful error messages.

VI.3 Protein List View

- You must display all ligands listed in the `ligands.txt` file (see attachments folder). This file contains a list of ligand identifiers, one per line.
- The list must be displayed in a scrollable view (UITableView on iOS, RecyclerView on Android, or equivalent).
- You must implement a search bar that enables users to filter the ligand list in real-time. The search should:
  - Filter as the user types (no need to press a search button)
  - Be case-insensitive
  - Search through ligand identifiers
  - Update the list immediately as the search query changes
- When a user selects a ligand from the list, the app must:
  - Display a loading indicator (spinning wheel, progress bar, or skeleton screen)
  - Fetch the .cif file from RCSB: `https://files.rcsb.org/ligands/view/{ligand}.cif`
  - Parse the .cif file to extract atomic coordinates and bond information
  - Navigate to the Protein View to display the 3D model

- If the ligand cannot be loaded, you must display a warning popup/alert with a clear error message explaining what went wrong:
  - "No internet connection. Please check your network."
  - "Ligand not found (404). This ligand may not exist in the database."
  - "Failed to parse ligand data. The file may be corrupted."
  - "Request timeout. Please try again."
- The loading indicator must be visible during the entire loading process and must disappear once loading is complete (success or failure).
- The list should handle large datasets efficiently without lag or stuttering.

VI.4 Protein View

- For this section, you may use:
  - iOS: SceneKit (recommended) or Metal
  - Android: ViroCore, filament, or OpenGL ES
  - Multiplatform: Any appropriate 3D rendering library
  - **Note**: Full game engines (Unity, Unreal) are NOT allowed. You must integrate 3D rendering into a standard mobile app.
- Display the ligand model in 3D with the following requirements:
- **CPK Coloring**: You must use the standard CPK (Corey-Pauling-Koltun) color scheme for atoms:
  - Carbon (C): Black or Gray
  - Hydrogen (H): White
  - Oxygen (O): Red
  - Nitrogen (N): Blue
  - Sulfur (S): Yellow
  - Phosphorus (P): Orange
  - Other elements: Use standard CPK colors (easily found online)

- **Ball-and-Stick Model**: You must represent the ligand using the Ball-and-Stick model:
  - Atoms are represented as spheres (balls)
  - Bonds are represented as cylinders (sticks) connecting atoms
  - Sphere sizes should be proportional to atomic radii (or uniform for simplicity)
  - Bond cylinders should be thinner than atom spheres
- **Atom Information**: When a user taps/clicks on an atom, display information about it:
  - Show a tooltip, popup, or overlay
  - Display at minimum: the atom's element symbol (C, H, O, N, etc.)
  - Optionally: atom name, residue information, coordinates
  - The tooltip/popup should disappear when the user taps anywhere else on the screen or taps another atom
- **Gesture Controls**: The user must be able to interact with the 3D model:
  - **Rotation**: Drag/swipe to rotate the molecule around its center
  - **Zoom**: Pinch gesture to zoom in/out
  - **Pan**: Two-finger drag to pan the camera (optional but recommended)

  All gestures should be smooth and responsive (60 FPS target).
- **Share Functionality**: Include a "Share" button that allows users to:
  - Capture a screenshot of the current 3D view
  - Share it via the platform's native share sheet
  - Share to social media, messaging apps, email, or save to photos
- **Camera and Lighting**: Set up appropriate camera positioning and lighting so the molecule is clearly visible and aesthetically pleasing:
  - Initial camera position should show the entire molecule
  - Lighting should highlight the 3D structure (avoid flat appearance)
  - Consider using multiple light sources for better depth perception
- The 3D rendering must be smooth and performant. No lag, stuttering, or frame drops during interaction.

Chapter VII

Bonus

The bonus part will only be evaluated if the mandatory part is **PERFECT**. Perfect means that the mandatory part has been fully implemented and works without any malfunctions. If you have not completed **ALL** of the mandatory requirements, your bonuses will be **TOTALLY IGNORED**.

Consider implementing these bonus features to enhance your application. Each bonus should be fully functional and add real value to the user experience:

VII.1 Multiple Visualization Models

Implement additional molecular representation models beyond the mandatory Ball-and-Stick model. Users should be able to switch between different visualization modes:

- **Space-Filling Model (CPK Model)**: Atoms are represented as overlapping spheres with sizes proportional to their van der Waals radii. This shows the molecule's actual volume and surface.
- **Wireframe Model**: Display only the bonds as lines, without atom spheres. This provides a clearer view of the molecular structure for complex molecules.
- **Stick Model**: Similar to ball-and-stick but with very small or no spheres, emphasizing the bond structure.
- Include a UI control (segmented control, dropdown, or buttons) to switch between models in real-time without reloading the molecule.

VII.2 Advanced User Interface

Enhance the visual design and user experience of your application:

- **Custom Table/List Cells**: Design beautiful custom cells for the ligand list with icons, previews, or additional information.

- **Smooth Animations**: Add polished transitions between views, animated loading states, and micro-interactions.
- **Dark Mode**: Implement a complete dark mode theme that works throughout the app, including the 3D view with appropriate lighting adjustments.
- **Onboarding**: Create an onboarding flow for first-time users explaining how to use the app and its features.
- **Settings Screen**: Add a settings view where users can customize preferences (default visualization model, color schemes, etc.).

VII.3 Enhanced Molecular Interactions

Add advanced interaction features for exploring molecular structures:

- **Atom Highlighting**: When hovering over or selecting an atom, highlight all atoms of the same element type.
- **Bond Information**: Display bond types (single, double, triple) and bond lengths when tapping on bonds.
- **Measurement Tools**: Allow users to measure distances between atoms or angles between bonds.
- **Atom Labels**: Option to display element symbols directly on atoms in the 3D view (toggleable).
- **Center on Atom**: Double-tap an atom to center the camera on it with a smooth animation.

VII.4 Performance and Caching

Optimize the application for better performance and offline capability:

- **Local Caching**: Cache downloaded .cif files locally so previously viewed ligands can be accessed offline.
- **Lazy Loading**: Implement efficient lazy loading for the ligand list if it contains many items.
- **Background Parsing**: Parse .cif files on a background thread with progress indication.
- **Memory Management**: Implement efficient memory management for large molecules, possibly using level-of-detail (LOD) techniques.
- **60 FPS Guarantee**: Ensure smooth 60 FPS rendering even for complex molecules with many atoms.

VII.5 Extended Sharing and Export

Enhance the sharing capabilities of your application:

- **Custom Share Messages**: When sharing, include a custom message with the ligand name, number of atoms, and molecular formula.
- **Multiple Export Formats**: Allow exporting the view as different image formats (PNG, JPEG) or even as a 3D model file.
- **Video Recording**: Record a short video of the molecule rotating and share it.
- **Favorites System**: Allow users to favorite ligands and maintain a favorites list for quick access.
- **Comparison View**: Display two molecules side-by-side for comparison.

Chapter VIII

Turn-in and peer-evaluation

Turn your work in using your `GiT` repository, as usual. Only work present on your repository will be graded in defense.

VIII.1 Evaluation Guidelines

During the peer evaluation, the following aspects will be assessed:

- **Functionality**: All mandatory features must work correctly without crashes, freezes, or errors.
- **Code Quality**: Your code should be well-organized, readable, and follow platform-specific best practices and conventions.
- **Error Handling**: The application must handle all error cases gracefully with appropriate user feedback.
- **User Experience**: The interface should be intuitive, responsive, and provide clear feedback for all operations.
- **Security**: Authentication must be implemented securely with no plain-text password storage.
- **Performance**: The application should run smoothly on real devices without lag or memory issues.
- **3D Visualization**: The molecular rendering must be accurate, using correct CPK colors and ball-and-stick representation.

VIII.2 Common Pitfalls to Avoid

- Failing to test on real devices (simulators/emulators may not reveal performance issues)
- Storing passwords in plain text or using weak hashing algorithms
- Not handling network errors or timeouts

- Blocking the main thread with network or parsing operations
- Memory leaks from improper 3D object management
- Not implementing the security requirement for the Login View
- Using deprecated .pdb format instead of .cif format
- Poor 3D rendering performance (< 30 FPS)

Remember: This project combines multiple complex domains (3D graphics, networking, security, bioinformatics). Start early, test frequently, and don't hesitate to ask for help on the forum or from your peers. Good luck!